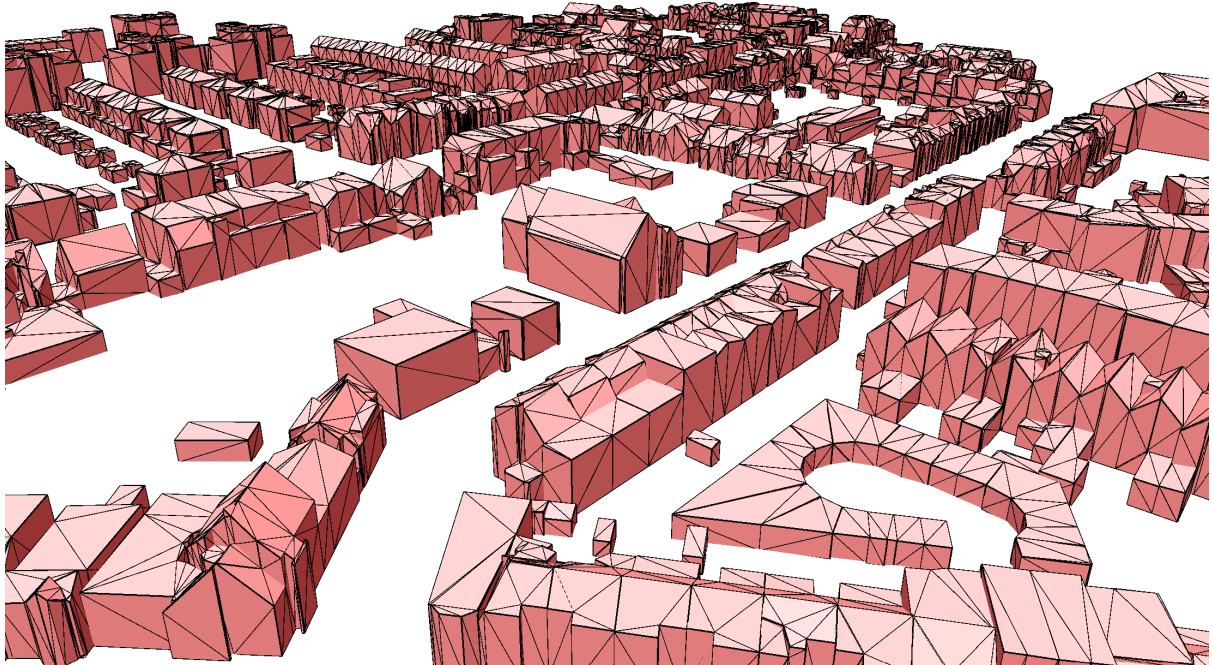


H2 202406107

GEO1004 HW2 — 3D city model processing

Arda Baysal (5484987) Daman Dogra (6407196) Tejas Ziegler (6575498)
June 2026



1 Introduction

We implement a pipeline to process tiles from the 3DBAG dataset, a nationwide 3D building model for the Netherlands, computing two new per-building attributes: enclosed volume (`geo1004_volume`) and total roof area (`geo1004_total_roof_area`). The implementation uses C++ with the CGAL library for geometric computations and `nlohmann/json` for CityJSON parsing. The pipeline is split across five modules, enabling parallel development on separate Git branches.

2 Methodology

Each team member owned a separate pipeline stage on their own Git branch, which meant some traversal logic was duplicated across modules, a deliberate trade-off to minimise merge conflicts and allow each member to independently familiarise themselves with the CityJSON structure.

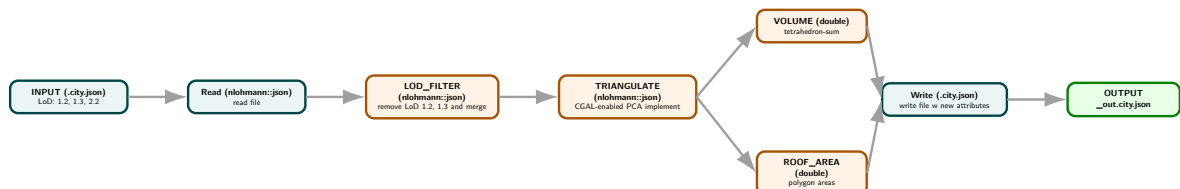


Figure 1: CityJSON Processing Pipeline.

2.1 LoD Filtering and Structural Normalisation

CityJSON stores geometry in a `Building/BuildingPart` hierarchy. A flattening algorithm consolidates all LoD 2.2 geometry into the parent `Building` and removes its children `BuildingPart` objects, producing a flat schema for volume calculation. For each `Building`, LoD 2.2 geometries are collected from both the parent and all children, merged into the parent, and the `BuildingPart` objects are deleted.

Engineering Decisions and Issues

- **Two-pass deletion:** `BuildingPart` IDs are collected first, then deleted in a second pass to avoid iterator invalidation.
- **Geometry at parent level:** Initially only `BuildingPart` children were searched, silently skipping buildings whose LoD 2.2 geometry was stored directly on the `Building`. Fixed by also checking the parent's own geometry array before iterating children.
- **Ghost children and schema cleanup:** A `.contains()` check guards against missing child references; `co.erase("children")` removes the stale array to pass `cjval` validation.

2.2 Triangulation of the surfaces

Algorithm: 2D Surface Triangulation

The triangulation step converts each LoD2.2 polygonal surface of every `Building` into triangular surfaces while preserving the CityJSON structure and semantics. For each LoD2.2 `Solid`, the program follows the hierarchy `shell` → `surface` → `ring` → `vertex` indices. The referenced vertices are converted to real-world coordinates using $\mathbf{p} = \mathbf{v} \cdot \mathbf{s} + \mathbf{t}$, where $\mathbf{v}$ is the stored vertex, and $\mathbf{s}$ and $\mathbf{t}$ are the CityJSON scale and translation vectors. A best-fitting plane is computed for each surface, and the 3D points are projected onto this plane to obtain a 2D polygon.

Projected ring edges are inserted as constraints in a CGAL constrained Delaunay triangulation; a domain-marking step from the infinite face classifies triangles as interior or exterior, and only interior triangles are kept. Each interior triangle is mapped back to the original CityJSON vertex indices and written into a new shell, with semantic values copied from the originating surface so that, for example, triangles from a `RoofSurface` remain correctly labelled for the roof-area calculation.

Engineering Decisions and Issues

- **Orientation preservation:** The original projected outer ring was used as the orientation reference by computing its signed 2D area. Each generated triangle was checked in the same way. If the signed areas had opposite signs, two vertices of the triangle were swapped before writing the triangle back to CityJSON. This preserved the orientation of the original surface in the generated triangles.
- **Degenerate and invalid cases:** Some surfaces contained problematic rings, such as repeated points, zero-length edges, or rings that collapsed after projection and cleaning. Before triangulation, consecutive duplicate points and repeated closing vertices were removed, for example `[A,B,B,C,D]` → `[A,B,C,D]`. If this reduced a ring to fewer than three usable vertices, then the ring could no longer define a polygonal surface and was not passed to the constrained triangulation.
- **Fallback and preservation of original surfaces:** When a surface could not be triangulated safely, the program avoided deleting it from the model. For simple four-vertex surfaces, a fallback split was used, creating two triangles from the quadrilateral. In other unsafe cases, the original surface was kept unchanged to avoid creating holes in the building geometry. In the final tested output, the kept surfaces still had triangular boundary structure, so no non-triangular surfaces remained.

2.3 Volume

Algorithm: Signed Tetrahedron Method

The assignment prescribes decomposing each building solid into tetrahedra and summing their signed volumes – a 3D extension of the shoelace formula. For every triangular face, a tetrahedron is formed with the coordinate origin `(0,0,0)` as apex, and its volume computed via the Scalar Triple Product:

$$V = \frac{1}{6}(\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})) \quad (1)$$

Because the volume is signed, faces pointing toward the origin contribute negatively and faces pointing away positively. When summed, the “air” between the origin and the building cancels, leaving only the true interior volume.

Engineering Decisions and Issues

- **Triangle fan over re-triangulating failed faces:** The triangulation step occasionally retains original polygons for near-zero-area faces. A triangle fan loop (pivoting on the first vertex) handles n-gons natively without a second triangulation pass.
- **Deferred `std::abs()`:** Applying `std::abs()` only at the final `CityObject` level keeps void subtraction correct and eliminates floating-point residuals (e.g. `-0.0000001`).
- **On-the-fly decompression:** CityJSON integer coordinates are decompressed to real-world metres immediately before calculation, preventing precision loss from large translation offsets.
- **Concave geometries:** The signed tetrahedron method handles concave shapes such as ‘C’ or ‘E’-shaped buildings correctly; by construction, the Divergence Theorem ensures coplanar interior walls cancel without double-counting.
- **Shell indexing and void handling:** A single `size_t i` counter adds `volume_shell` for `i == 0` (outer boundary) or subtracts for `i > 0` (void), correct regardless of winding order.
- **Validity-based filtering:** Buildings with a non-empty `b3_val3dity_lod22` attribute are skipped before volume computation, as the signed tetrahedron method requires a watertight mesh. This uses 3DBAG’s own quality signal rather than an external threshold.

2.4 Total area of the RoofSurfaces

The implementation of `add_total_roof_areas()` iterates over every `Building` in `CityObjects`, and for each `Solid` geometry walks the nested boundary structure, using the parallel `semantics.values[i][k]` index to identify surfaces of type `RoofSurface`. A check on `ring.size()` (`ring.size() != 3`) skips any non-triangle surface. The real-world coordinates of each vertex are recovered and area of each triangle is computed as

$$A = \frac{1}{2} \| (\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_3 - \mathbf{p}_1) \|,$$

summed across all `RoofSurface` triangles and written as `geo1004_total_roof_area`.

3 Experiments and results

3.1 Triangulation

Both the input and output files pass `cjval` schema validation with no errors. The warnings present in both files — duplicate vertices, unused vertices, and extra root properties — originate in the source 3DBAG data and are unchanged by the pipeline, confirming that no new topological issues were introduced during processing.

Warning	Input (9-284-556.city.json)	Output (9-284-556_out.city.json)
Duplicate vertices	1857+	1857+
Unused vertices	37156	37156
Extra root properties	present	present
Errors	none	none

Table 1: `cjval` validation results before and after processing.

3.2 Volume

Accuracy was evaluated by comparing `geo1004_volume` against `b3_volume_lod22` on two tiles (Figure 2 and 3). After excluding buildings with non-empty `b3_val3dity_lod22` — flagged as geometrically invalid by 3DBAG itself — both tiles achieve a mean signed error of 0.00 m^3 and median relative error of 0.01 %, indicating no systematic bias.

Prior to filtering, tile 9-284-556 showed a mean relative error of 124 % against 9-588-304’s 0.08 %. Edge-manifold analysis confirmed that every outlier building had open shells (6–14 boundary edges) in the input CityJSON — predating the pipeline entirely. All such buildings carry `b3_kwaliteitsindicator: false` in the source data.

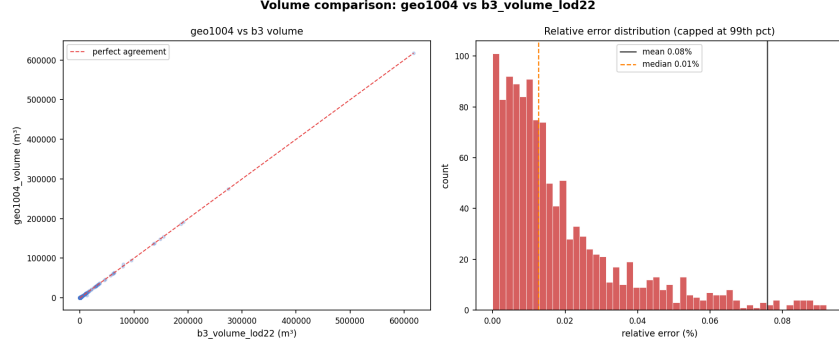


Figure 2: Volume comparison for tile 9-284-556. Left: computed vs. reference volumes; points closely follow the perfect agreement line. Right: relative error distribution, median 0.01%.

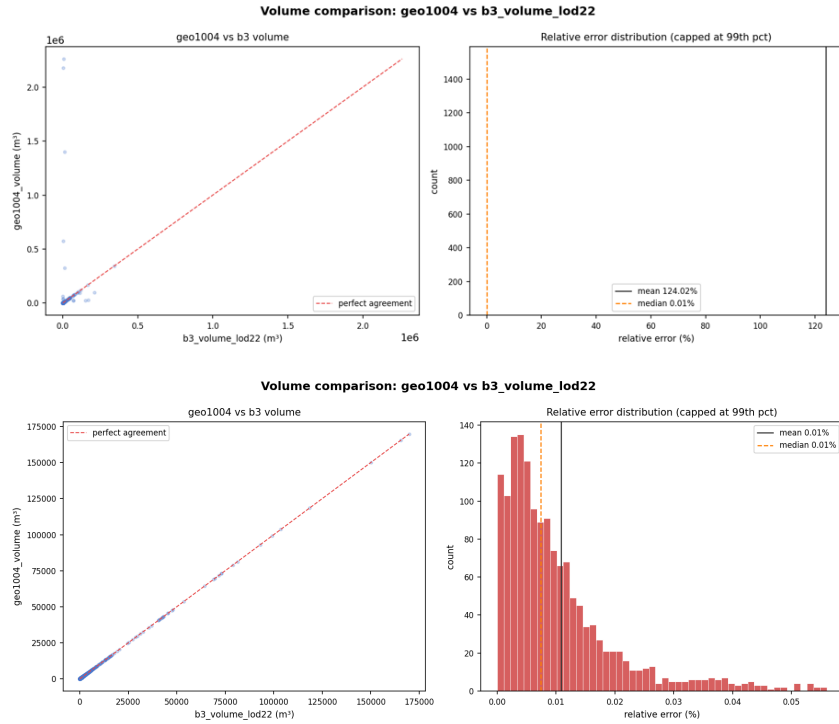


Figure 3: Volume comparison for tile 9-284-304 before `b3_val3dity_lod22` filter (top) and after (bottom), excluding buildings flagged by `b3_val3dity_lod22`. Outlier buildings with open shells in the source data inflate the mean relative error from 130.82% to 0.03% after filtering; the median remains 0.01% in both cases, confirming the pipeline is correct for all valid input.

3.3 Validation on synthetic datasets

For a rectangular box with footprint $l \times w$ and height h : $V_{\text{box}} = lwh$, $A_{\text{box}} = lw$. For a gabled roof of ridge height r placed on a box of wall height h_w :

$$V_{\text{gabled}} = lwh_w + \frac{1}{2}lwr, \quad A_{\text{gabled}} = 2l\sqrt{\left(\frac{w}{2}\right)^2 + r^2}.$$

The hip and L-shaped cases are decomposed into the same primitives (boxes and triangular prisms), and their analytical values were computed accordingly.

Using these analytical ground-truth values, we generated synthetic cities (seeded by 7, 13, 42, 69) with known roof area and volume (Figure 4), input them into the pipeline, and compared LoD2.2 attributes between the originals and output files. All buildings matched within 10^{-2} accuracy. These tests verify correctness under ideal conditions: mutually disjoint, manifold, and easily triangulated geometries.

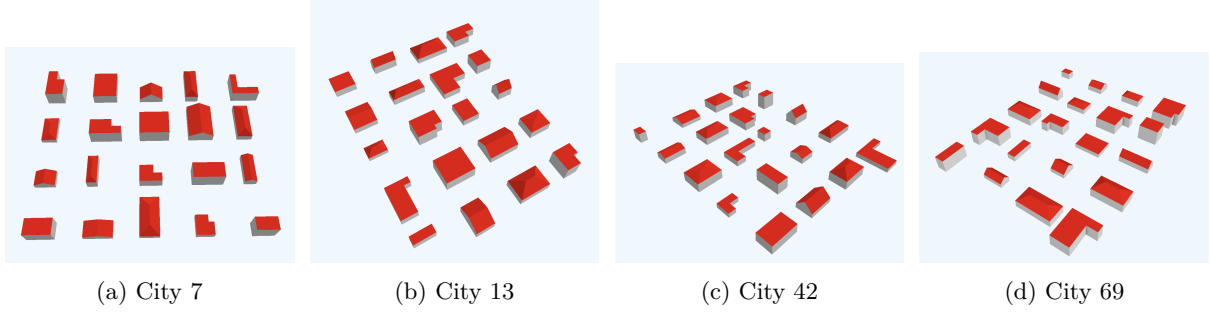


Figure 4: The 4 synthetic cities used to test volume and roof area, visualised on ninja.cityjson.org. These include *flat box*, *gabled*, *hip* and *L-shaped* buildings.

4 Discussion and conclusion

The pipeline produces valid CityJSON 2.0 output and meets the accuracy requirements on all valid input. The principal limitation is robustness on geometrically invalid input: the 3DBAG contains buildings that are not fully manifold or have degenerate surfaces, which the triangulation step handles defensively by skipping rather than aborting. A second limitation is performance — the pipeline processes the full tile sequentially, acceptable for a single tile but not for batch processing. Future work could address both by adopting a repair pre-processing step (e.g. using `val3dity` output to guide targeted fixes) and parallelising per-building computations.

Once invalid buildings are excluded using 3DBAG’s own validator flags, both tiles perform identically. A notable data quality issue is that `b3_volume_lod22` is populated even for buildings that 3DBAG marks invalid, likely because 3DBAG computes volumes from a watertight internal representation before exporting geometrically degraded CityJSON, making it an imperfect ground truth for invalid buildings. A remaining limitation is that skipped buildings do not produce a `geo1004_volume` attribute in the output; a more informative approach would write `null` alongside a boolean validity flag.

5 Work Allocation and AI use

5.1 Contributions

Arda Baysal: LoD filtering and Triangulation. **Daman Dogra:** Bug fixes in LoD filtering. Volume calculation and analysis. **Tejas Ziegler:** Roof area calculation; managing repository structure.

The writing of the report was done collectively. Each member contributed the most to the writing of their respective part. The analysis and validation steps were also discussed together.

5.2 GenAI statement

LLMs (Claude Opus 4.7, Sonnet 4.6) were used to advise on report wording and presentation and to aid in code implementation and debugging (C++ syntax when needed). Some functions that helped in debugging and locating issues were written with the help of LLMs. Also used to implement a synthetic city-generation script, whose analytical formulas were all verified. All content and code produced by such LLMs were reviewed, understood, and implemented our own modifications.